



2024865008 변정훈

GIT





GIT이란?

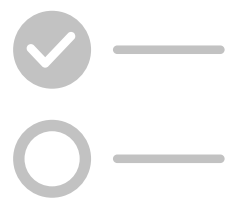
GIT이란 소스 코드의 변경 이력을 기록하고 협업을 지원하는 분산형 버전 관리 시스템입니다.

완벽한 이력 추적 (Version Control): 파일의 생성, 수정, 삭제 등 모든 변경 과정을 시간순으로 기록하며, 문제 발생 시 과거의 특정 시점(버전)으로 즉시 복구할 수 있습니다.

분산형 아키텍처 (Distributed): 중앙 서버에만 의존하지 않습니다. 오프라인 상태이거나 중앙 서버가 다
운되어도, 각 개발자의 컴퓨터(로컬)에 전체 코드와 이력이 복제되어 있어 중단 없이 작업이 가능합니다.

안전한 병렬 개발 (Branch & Merge): 원본 코드에 영향을 주지 않고 새로운 기능을 개발할 수 있는 평행
우주(브랜치)를 쉽게 만들고, 검증이 끝나면 원본에 안전하게 합칠 수 있습니다.

데이터 무결성 (Integrity): 모든 변경 사항은 고유한 해시(SHA-1) 값으로 관리되어 데이터 유실이나 파일
손상을 즉각적으로 감지합니다.





GIT이란?

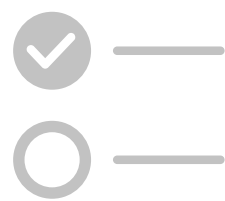
GIT이란 소스 코드의 변경 이력을 기록하고 협업을 지원하는 분산형 버전 관리 시스템입니다.

완벽한 이력 추적 (Version Control): 파일의 생성, 수정, 삭제 등 모든 변경 과정을 시간순으로 기록하며, 문제 발생 시 과거의 특정 시점(버전)으로 즉시 복구할 수 있습니다.

분산형 아키텍처 (Distributed): 중앙 서버에만 의존하지 않습니다. 오프라인 상태이거나 중앙 서버가 다운되어도, 각 개발자의 컴퓨터(로컬)에 전체 코드와 이력이 복제되어 있어 중단 없이 작업이 가능합니다.

안전한 병렬 개발 (Branch & Merge): 원본 코드에 영향을 주지 않고 새로운 기능을 개발할 수 있는 평행 우주(브랜치)를 쉽게 만들고, 검증이 끝나면 원본에 안전하게 합칠 수 있습니다.

데이터 무결성 (Integrity): 모든 변경 사항은 고유한 해시(SHA-1) 값으로 관리되어 데이터 유실이나 파일 손상을 즉각적으로 감지합니다.





GIT 명령어

구성	git - v	브랜치 (Branch)	git branch <브랜치>
초기화(Init)	git remote - v	당겨오기 가져오기	git pull <브랜치> git fetch <브랜치>
추적(Track)	git status	되돌리기 (Revert)	git revert
버전 생성 (Commit)	git commot - m	병합(Merge)	gir merge <브랜치>
확인(Log)	git show <브랜치>	재배치 (Rebase)	git rebase



이 외에도 태그, 선별 적용 등 여러 카테고리의 항목들이 존재합니다.



GIT의 핵심 기능

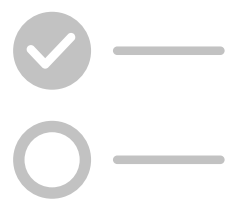
Branch, Revert

Merge

브랜치(Branch)는 현재의 코드 상태에서 가지를 뺀어 사용자만의 독립적인 작업 공간을 만드는 기능입니다. 브랜치를 만들면 원본을 건드리지 않고 새로운 알고리즘을 테스트하거나, 구조를 수정할 수 있습니다. 코드를 완전히 망쳐도 원본에는 영향이 없습니다.

리버트(Revert)는 이미 저장된 과거의 특정 작업을 취소하는 새로운 커밋을 추가하는 기능입니다. 기록을 지워버리는 reset과 달리, revert는 기록을 남기면서 작업을 취소하기 때문에 함께 작업 중인 다른 팀원들에게 피해를 주지 않으면서 잘못된 코드를 안전하게 수습할 수 있습니다.

머지(merge)는 나누어서 작업한 결과물들을 다시 하나로 합치는 과정입니다. 브랜치를 사용하여 개발을 진행한 후, 각자의 작업물을 다시 하나로 합치는 과정이 필요한데, 이러한 과정을 merge 라고 합니다.





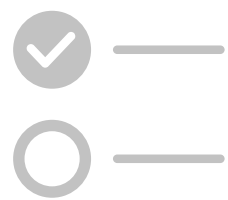
GIT이 왜 필요할까?

GIT을 사용한다면 다수의 인원과 함께 개발을 진행할 때 발생할 수 있는 혼란스러운 상황들이 발생하지 않도록 관리할 수 있습니다.

프로그램_진짜_찐막 처럼 많은 오류 수정을 거치면서 생기는 많은 파일 생성을 방지할 수 있습니다. GIT이 내부적으로 모든 변경 사항(버전)을 기록하고 있기 때문에, 파일을 수정하더라도 파일명을 수정할 필요가 없으며, 언제든지 과거의 특정 시점으로 되돌아갈 수 있습니다.

안전한 "평행우주"에서 테스트가 가능합니다. 잘 실행되는 코드를 개선하려다가 오류가 나서 원래 파일로 돌아가지 못하는 불상사를 방지할 수 있습니다. GIT의 기능 중 하나인 브랜치를 사용하여 원본 코드를 안전하게 둔 상태로 다양한 기능을 테스트할 수 있습니다. 코드 개선을 성공했다면 원본에 합치고, 개선을 실패했거나 변경점이 없다면 브랜치를 지워버리면 끝입니다.

또한 GIT은 모든 변경 사항마다 커밋 메시지를 남기게 합니다. 그렇기 때문에 누가, 언제 코드를 수정했는지 확인이 가능하기 때문에 팀 프로젝트를 진행할 경우 큰 도움을 줍니다.





감사합니다